

Techniques for Creating Process Injection Attacks with Advanced Return-Oriented Programming

DEF CON 32

Las Vegas, Nevada, August 2024

Bramwell Brizendine and Shiva Shashank Kusuma¹

¹ University of Alabama in Huntsville, Huntsville, AL 35810, USA
bramwell.brizendine@gmail.com



Abstract. This white paper presents advanced techniques for process injection using return-oriented programming (ROP), addressing the significant hurdles posed by chaining numerous WinAPIs and native APIs with complex parameters and return values. We offer practical solutions to manage this sometimes delicate and complex process, including a novel, universal method for reliable string comparison via ROP to accurately identify PIDs of target processes, thereby being able to inject into them. By developing multiple patterns for different APIs, particularly those that favor an approach based on using the pushad instruction, and documenting variations that necessitate the "sniper" approach, we provide a methodology for repeated use. Our research aims not only to demonstrate process injection via ROP but also to create and provide reliable and reusable templates for using WinAPIs in similar future efforts.

Keywords: Return-oriented Programming, Process Injection

1 Introduction

Return-oriented programming (ROP) has been celebrated as an immensely powerful technique for low-level software exploitation, enabling attackers to execute shellcode even in spite of otherwise very powerful mitigations, such as DEP. This white paper explores the methodology of process injection with advanced ROP. We provide details on the mechanics of utilizing ROP to facilitate process injection. With process injection, as an advanced exploitation technique, performing this completely via ROP is perhaps unprecedented, to the extent done here where nearly 30 primary and auxiliary APIs are used entirely via ROP. This paper serves to provide information on how process injection may be conducted purely through the usage of ROP.

This paper showcases techniques for process injection using advanced ROP. Process injection via ROP offers significant hurdles, requiring many WinAPIs to be chained together, each with complex parameters and return values. Here, we provide practical details on how to best manage this somewhat delicate process.

One seemingly insurmountable challenge is identifying the target binary, as string comparison can be extremely difficult in ROP due to the potential lack of needed ROP gadgets. We unveil a unique, universal solution that provides a reliable means of string comparison via ROP, which consistently allows a specific process to be pinpointed and injected into.

We have created numerous patterns for different WinAPIs, allowing for as many as a dozen ways of preparing a specific WinAPI via ROP, particularly when using an approach centered around the pushad instruction. Some WinAPIs, however, offer zero patterns for pushad, necessitating reliance on the much-lauded "sniper" approach, i.e. the mov dereference technique. This paper documents all such variations of patterns for the WinAPIs involved in our demonstrated process injection. The goal of this research is not merely to demonstrate a one-off example of process injection via ROP, but to provide a methodology that can be reused time and time again, offering original templates for others to use the same WinAPIs or native APIs when attempting process injection via ROP.

2 Background

2.1 Process Injection

Process injection is an advanced technique used to allow executable code to be injected into the address space of a target process, ideally without being detected by others. Once present, the goal is to try to further evade detection from EDR or other mitigations. These solutions typically will look at heuristics on programmatic behavior as well as analyzing the code within processes. Injecting code into a legitimate process potentially can be highly beneficial, as the permissions and context of the victim process may be used. This may serve to potentially evade some security measures, if the external process injected into is trusted.

2.2 MITRE ATT&CK Framework and ROP

The MITRE ATT&CK framework is a highly detailed knowledge base for adversary techniques and tactics, primarily based on experience with real-world incidents. This has become a widely used resource, and it seeks to provide classification of different techniques into various categories, allowing security professionals to have a common reference point. With numerous techniques and sub-techniques, each is documented in abundant detail. MITRE documents numerous aspects of the threat landscape, including initial access, execution, persistence, privilege escalation, defense evasion, credential access, discovery, lateral movement, collection, exfiltration, and command and control. Included among these is process injection. MITRE ATT&CK framework classifies process injection under the T1055 tactic, complete with countless techniques and sub-techniques.

Some process techniques are as follows:

DLL Injection (T1055.001). This technique features a Dynamic Link Library (DLL) being injected into the address space of another process. This can be done with various WinAPIs or native APIs.

Process Hollowing (T1055.012). This technique allows a legitimate process to be selected and then essentially replaced with malicious content. The original process will be placed into a suspended state, with its original code being removed or hollowed out. Given that the original process will have been hollowed out, this will permit the attacker to execute their injected code within the context of the original process, which as a legitimate process likely may go undetected by antivirus.

Portable Executable Injection (T1055.002). This technique involves code being injected into the memory of an active process. The injected code is usually either shellcode or a PE file. Once injected, the author can then trigger it to be executed. This method may use a variety of API calls such as VirtualAllocEx, WriteProcessMemory, and CreateRemoteThread.

Thread Execution Hijacking (T1055.003). With this technique, code can be injected into a process. An attacker can hijack execution of an already existing thread contained in the process. In order to do this, one needs to manipulate the thread's context, such that its execution flow is redirected to the injected code. Execution of threads within a victim process can be facilitated with the use of APIs like SetThreadContext and GetThreadContext.

Asynchronous Procedure Call (APC) Injection (T1055.004). This method utilizes the Asynchronous Procedure Call (APC) mechanism in Windows to inject code. By queuing an APC to a thread in the victim process, injected code can be executed once the thread enters an Alertable state.

2.3 Process Injection via ROP

Process injection via ROP has only had limited usage degree previously. We are not aware of any attempts where there has been a security vulnerability, EIP has been captured, and then process injection was done entirely via ROP. When it is used, it is invariably done in the context of malware or red-team software that is experimenting with process injection. That is to say, these are efforts predominately written in a higher level language, with some portions in Assembly and indeed some portions have one or more APIs called via ROP.

Banking malware Dridex utilized ROP as part of its process injection routine [1]. With Atom bombing, malicious code can be written into a global atom table. An APC called to the APC queue of the victim process can then be made with an appropriate API. Once done, it can then call the `GlobalGetAtomName` function, which gets the malicious code from the global atom table, allowing it to then be inserted into the memory of victim processes. Using atom tables means that the injected code will not be ready to be executed. As part of some techniques, ROP is then used – in a highly limited fashion – to change the memory region to RWX – thereby allowing the injected code to run. `NtQueueApcThread` can be used to cause the victim process to use `NtSetContextThread`, initiating a ROP chain that creates RWX memory. The ROP chain can then copy the payload from a Read-Write (RW) section of the process to the RWX allocation, leading to its execution.

Stackbombing [2] can also be used to provide malicious ROP chains, which can be inserted and then caused to execute. The current stack is backed up using `memmove`. Once it is suspended (`SuspendThread`) and then put in an altertable state, this technique involves modifying the stack to point to an unused section where the ROP chain will have been injected. `GetThreadContext` is used to capture CPU context and get the return address. The return address can then be overwritten. When `ResumeThread` is then called, it then allows it start executing into the injected ROP. Thus, this can allow the execution flow of the thread to be hijacked, thereby executing the inserted payload. Additionally, the stack can be restored and allow execution to continue. Stackbombing has been effective historically at bypassing mitigations. Some limitations with this have been that it just got the ROP to execute, not any shellcode beyond that, but further work could prove otherwise.

Pinjectra is a C++ library that can be used to implement different process injection techniques, created by Itzik Kotler and Amit Klein, as part of their Black Hat talk in 2019 [3]. Pinjectra makes use of stackbombing, and as such it implements ROP. Work by Omer Yair extends what was done [4]. For instance, Omer Yair demonstrated that in his *ROP – From Zero to Nation State in 25 Minutes* talk, where advanced ROP can be done to help avoid detection, even by calling parts of Windows syscalls via ROP gadgets, e.g. calling the syscall / ret portion of the 64-bit `NtYieldExecution` Windows syscall.

Ghost-writing [3] provides process injection by manipulating process memory with `SetThreadContext` to set registers, with the thread being left to run in an infinite loop, and a combination of ROP with `NtQueueApcThread` and `memset`, followed by additional ROP operations with ROP to restore volatile registers and resume normal execution.

ROP can also be used by malware as a means of obfuscation, allowing malicious content to be split into different chunks, allowing for covert distributed execution. This could be done via ROPE [5], which facilitates multi-process malware execution being done in conjunction with ROP.

3 Shellcodeless Return-Oriented Programming (ROP)

Previous efforts often combine ROP with higher-level languages, rather than performing pure ROP once a vulnerability is weaponized. Intermixing ROP with code written in higher-level languages, like with Pinjectra, Yair's extension of Pinjectra, or Ghost-writing, can be highly effective, but it is outside the scope of what this research focuses on. For instance, custom malware or red-team software might handle some of the work related to process injection by invoking ROP as needed. This differs considerably from overcoming a vulnerability and operating entirely within ROP. Our research focuses on compromised, vulnerable binaries where the payload consists solely of ROP gadgets responsible for the full process injection attack. While custom malware or red-team software can use ROP to avoid EDR detection, intermixing ROP with higher-level languages is, to reiterate, outside the scope of this research. The goal here is to conduct the entire process injection, from start to finish, purely via ROP, without any additional help from libraries, emphasizing the mechanics of performing these actions through pure ROP. With this research, we embrace and to extend shellcodeless ROP, taking it to a level that likely probably has not been done before in the Windows ecosystem – in terms of complexity and the sheer multitude of components and various APIs utilized.

3.1 Introduction to Shellcodeless ROP

By reimplementing shellcode through pure ROP, we eliminate the need to bypass Data Execution Prevention (DEP) and achieve functionality equivalent to traditional shellcode execution. This approach involves directly invoking the desired WinAPIs or native APIs with correct parameters, each loaded via ROP. In no way is other help given for any aspect of this outside of what can be done through ROP.

In shellcodeless ROP, various WinAPIs are chained together, much like ROP gadgets, to produce complex functionality akin to that of a shellcode. Often a goal with exploitation is to overcome DEP, and execute the shellcode, which is the method where the attacker can perform arbitrary computation. That is, they introduce Assembly instructions that are executed, calling API functions as need be to achieve complex functionality. Although this technique has been existed with respect to Linux, its implementation on Windows platforms has been extremely rare due to the highly complex setup required. Usually in Windows, at most we have is two or three WinAPIs working in tandem. ROP ROCKET [6] is able to fully automate some shellcodeless ROP attacks, and that is also the origin of this research for ROP efforts into process injection, as it was originally going to be included in that tool – i.e. have the ROP chain generation be fully automated.

Related to advanced shellcodeless attacks in Windows, Brizendine also presented his research with Babcock on shellcodeless JOP in Windows at a past Black Hat Middle East in 2021, then called @Hack, giving an elaborate demo on shellcodeless JOP [7]. The Black Hat presentation was notable for having showcased the potential of this method. Their attack allowed for the download and execution of an executable file without the need for traditional shellcode. The implications of such are of course obvious.

3.2 Techniques for Shellcodeless ROP

Two primary approaches can be utilized for shellcodeless ROP: the pushad method and a sequence of mov dereferences, also sometimes called the sniper approach.

Pushad Method: In this approach, registers are preloaded with specific values. After the pushad instruction, these values are aligned on the stack in the correct order as required by the API to be called. This method uses the predefined order in which pushad pushes register values onto the stack, ensuring that the parameters are correctly set up for the subsequent API call.

Mov Dereferences: This technique involves precisely setting up the stack with parameter values through a series of mov instructions. By carefully arranging these values, the stack is prepared in such a way that the API can be called with the correct parameters already in place.

Both approaches require precise control over the stack and register states, so that the desired APIs are invoked with the correct parameters. This could require careful advanced planning.

4 Introduction to ROP Patterns

ROP patterns are structured sequences of gadgets designed to achieve a specific outcome by manipulating execution flow, which occurs directly after pushad. In a given ROP pattern, various registers are loaded with values that correspond to return address, parameters, runtime address, and more. With shellcodeless ROP, different patterns can essentially be chained together, allowing for multiple WinAPIs to function in unison; this facilitates the execution of complex tasks that would usually require shellcode to be achieved – all done without the use of traditional shellcode.

Slot 1	Slot 2	Slot 3	Slot 4	Slot 5	Slot 6	Slot 7	Slot 8
EDI	ESI	EBP	ESP	EBX	EDX	ECX	EAX

Fig. 1. Example of slot order for ROP pattern.

This pattern represents a typical setup where each register has been assigned a specific role or task. For instance, EDI may be used to store the return address, ESI could function as a ROPNOP (a RET instruction that does nothing), while other registers may be loaded with parameters or values pertinent to the execution of a particular API function.

4.1 # Chaining WinAPIs Together

An example pattern, one of a dozen, to execute the GetProcAddress WinAPI follows:

Register	Value
EAX	lpProcName (OpenProcess) [excluded regs: ECX]
ECX	hModule
ESP	SKIP
EDI	RET 8 [excluded regs: EAX]
ESI	ROPNOP (RETN)
EBP	PTR to GetProcAddress
EBX	JMP dword ptr [EBP]
EDX	ReturnAddress

Fig. 2. This is a pattern for GetProcAddress

In this chain, we can see the registers are loaded in the following manner:

- EAX holds a pointer to the procedure (function) name, lpProcName.
- ECX holds the handle to the module, hModule.
- EBP is a pointer that points to the GetProcAddress runtime address.
- EBX jumps to the address contained in the pointer at EBP.
- EDI and ESI deal with control flow.
- EDX holds the return addresses.

By having the values loading into their respective registers, when we do pushad, these values will be pushed onto the stack in a predictable order. By virtue of ROP, it will cause GetProcAddress to be called according to standard calling convention, with the return address and parameters on the stack in their proper location. This is achieved with a dereferenced jump the register containing a pointer that holds the runtime address of GetProcAddress. While pointers such as this may exist on the binary, the runtime address for a WinAPI cannot be foreseen, due to ASLR; thus, the pointers are immensely useful. Once a WinAPI is done, the return address becomes accessible via EAX, and other output can be provided from native APIs via OUT parameters (for output).

4.2 Creating Unique Patterns for ROP

Creating unique ROP patterns involves the strategic use of instructions like pushad, which pushes the values contained in registers onto the stack in a predictable order. This was originally done with functions like VirtualAlloc and VirtualProtect, going back to tools like Mona, and for each of these, two distinct patterns were documented. Having multiple patterns provided more options when doing automatic ROP chain generation. Though virtually never have patterns for others WinAPIs been thoroughly documented – typically, only one pattern, e.g. a specific pattern for one single use of LoadLibraryA, might be documented in a ROP chain that utilizes it. In fact, there are about a dozen patterns for LoadLibrary. In fact, we have never seen multiple patterns (i.e. more than one pattern per API), documented for any WinAPIs other than the above.

Automatic ROP chain construction tools are extremely rare. In Windows, the oldest and perhaps most well-known example is Mona. One of the most advanced tools for this purpose is ROP ROCKET [8]. This tool facilitates the creation of multiple ROP patterns, providing numerous fallback options if a particular pattern is unsuccessful. Given that some WinAPIs have as much as a dozen potential patterns, having multiple options like this is highly relevant for successful exploitation. Historically there has been no attention paid to patterns for WinAPIs used in ROP, just because there are so few mature ROP tools that utilize patterns. There does not seem to be much reason to develop multiple patterns if not to help automatic ROP chain generation. During manual

exploitation, one pattern is sufficient; it is not necessary to generate eleven additional patterns. This hinders the security community, as these patterns are not documented, and thus if someone wishes to use an API, they have to figure out how to do it. We are not aware of any documentation whatsoever on how to generate patterns, so discovering how to create a pattern potentially could require a not insignificant effort from an individual. Creating a pattern – and knowing how to do so – typically would require someone with a much more sophisticated understanding of how ROP operates at a low level.

5 General ROP Techniques for Process Injection

Process injection typically may require numerous functions, and thus more elaborate setup is required than with more simple efforts, such as a call to `VirtualProtect`. Some of the following advanced ROP techniques offer an effective means to manipulate and orchestrate the necessary ROP. Accordingly, this section explores advanced methods such as using the output of one function as a parameter for another, handling functions that take complex structures as parameters, and carefully storing important addresses and values, to avoid redundant work. These techniques help streamline our exploit process, reducing the number of gadgets or even APIs needed, and allowing for a greater degree of precision.

5.1 Using Function Output as a Parameter

One method is the utilization of the output from one function as a parameter for another API. For instance, the `GetProcAddress` WinAPI requires a handle to a module as its parameter. With the benefit of higher-level languages, this could much more easily be obtained. In ROP, we deal with just Assembly instructions, and calls to an API must be made tediously with sometimes not insignificant labor. There are no shared resources available. Thus, the only option in many cases is a call to an API that may provide a needed handle. That may solve the requirements, but in some cases, that may need to be fed to another API to get yet another handle, so on and so forth. With this particular example, it is simple, and the needed handle can be had from `LoadLibraryA`, which returns the handle to `EAX`. This value can then be transferred to other registers, as need be, until it is able to find its proper home in the pattern for the subsequent WinAPI.

5.2 Structures as Parameters

Some API functions will require pointers to structures as parameters. For instance, the `Process32First` function uses a structure that must be formatted correctly and populated with all the proper values. In such cases, the structure must be built in memory using ROP, most of the time. With this example, most of the structure, `PROCESSENTRY32`, is comprised of NULL bytes, so creating it is not exceptionally difficult. This can be done in a variety of ways, such as using `mov` dereferences, which is the method that would be favored for this approach.

The `AdjustTokenPrivileges` WinAPI is a significantly more complex example, requiring several nested structures. Each of these structures must be carefully plotted out and then painstakingly set up, one-by-one, via ROP, ensuring all needed pointers and values are in the correct locations. This could be a non-trivial, though not exceptionally demanding, task.

5.3 Storing Important Addresses and Values

With more advanced ROP, involving several WinAPIs, there will be a set of needed values that need to be saved, such as handles, runtime address, or even strings. Efficient ROP exploitation, accordingly, demands that we store important addresses or values in memory for subsequent later retrieval. For instance, obtaining the `hModule` from `LoadLibrary` is needed to get the base address of a DLL, which is also its handle. If it is not loaded, it will load it. If it is already loaded, then it will do nothing other than provide the handle, which is still needed in the ROP. That is, `GetProcAddress` requires an `hModule`. Given that there could be, for instance, numerous times a function from `Kernelbase` is used, is it necessary to keep calling `LoadLibrary` repeatedly for the same value? Thus, the convention of saving and then retrieving important values needed in a complex ROP chain is important. These can be saved anywhere in memory with RW permissions, to include the stack, the heap, newly allocated or mapped memory, etc. Though, care should be made when using the stack, to ensure that subsequent operations

do not cause values to be modified later. Storing important values in memory can save time and avoid (potentially numerous) redundant function calls to obtain the same value.

This approach mirrors the behavior of sophisticated shellcode, which frequently stores return values of API calls to better optimize execution. The same technique should be applied in shellcodeless ROP, where both addresses and other important values, such as handles, can be saved and retrieved as need be avoiding repetitive calls. Additionally, not only are these calls in some cases unnecessary, payload size is often a consideration, so not being wasteful with usage of available bytes is often essential. By following this guidance, multiple uses of the same value can be easily handled, with values being available in memory, at known and accessible locations, able to programmatically be retrieved as need be.

When dealing with saving and retrieving values, it is important to have some frame of reference to where in memory the value is, and to ensure that it is not lost. For instance, perhaps one register points to a table, and the 5th entry in that table will be a particular value. Thus, if this is maintained then one can use ROP to dynamically or programmatically calculate where it is via a series of gadgets. For instance, maybe at a particular point in the ROP, a register is 0x452 bytes from a table; at another location, it is 0x424 bytes, etc. It is a value that is a predictable, constant from that location in the ROP chain, but which will be different later in the chain, due to change in the stack with each gadget. These values may need to be calculated or recalculated as the ROP chain evolves. It is not necessary to use a table, as described, as that could add one additional layer of complexity. Even if the values are at random, safe locations on the stack, which are not subsequently overwritten, the above is applicable. With ROP ROCKET, emulation can be used to help calculate some of these types of changing distances. For manual exploit development, using a debugger and appropriate breakpoints would be necessary.

5.4 Combining Mov Dereference and Pushad

In advanced ROP chains, combining mov dereference and pushad techniques can form a highly effective pattern for process injection

Pushad is most effective for ROP chain patterns as it minimizes the number of bytes need. Mov dereference can allow the same to be achieved, but it can require significantly more gadgets. This of course is not always going to be the case, as in some cases, particularly with a very small sized binary, there may be insufficient attack surface, but in a majority of cases, pushad will be much more efficient at minimizing the number of bytes required. Utilizing pushad, up to four parameters can be loaded if avoiding the ESP register, or five parameters if using ESP. However, in a majority of cases, ESP will be problematic to use and must be avoided; this can limit a lot of possibilities. When combining these two approaches, the remaining parameters can be supplied directly through mov dereferences. This hybrid approach is preferred, when possible, because it often requires fewer bytes than using mov dereferences alone. This clearly makes it a more efficient option for when payload size is a consideration.

5.5 Supplying Strings Directly in the Payload

In some cases, it is possible to include strings directly within the payload. This method potentially could require careful handling of bad characters, such as NULL bytes, which can terminate strings prematurely. Given some binaries, these could be included directly. In other cases, the needed NUL bytes could be added later using ROP gadgets, with temporary placeholder values being used in their stead. This technique is not limited to strings; any data, such as pointers to values, can be supplied in this manner. When it is feasible to do so, supplying strings or other important non-strings data directly in the payload can be beneficial, as long as there are no EDR or anti-malware considerations. Clearly, in some cases, this may not be possible. As before, a common reference point can be found, and this could be used to calculate the distance to them. For instance, EDI might hold a location to one string, and that could be used as need be, and then adjusted to reach another string. When it is not needed, the common reference point could be stored via ROP and then subsequently retrieved as need be. Additionally, it is possible to provide some of these values or strings in an obfuscated form, such as having been XORed by a common value; that might evade some EDR measures. These could then be deobfuscated via ROP. However, doing this would require a lot of extra ROP gadgets, so it likely would only be practical if there were no payload size restrictions. The same few gadgets could be used time and time again for this process. Thus, by applying simple transformations to our strings or other data and reversing them during execution, what is being done with the strings or other values may be obscured to help evade detection.

6 Process Injection via ROP

With his research, we formed four complete ROP chains for complete process injection, though two were variations. In this paper, we will take a closer look at one of these, the most sophisticated one. This research is perhaps an anomaly as it does not try to provide a completely new process injection technique; if it did, it might try something like with fibers, going of the incredible work of Daniel Jary. Though whether the complexity of such could be done purely via ROP would remain to be seen. Instead, the focus is just on the mechanics of doing it via ROP and addressing the problems and issues that come up, to make it more practical. That includes the formation of patterns, addressing the string comparison problem, etc.

6.1 Case Study: Sample Process Injection

The below process injection technique, done purely by ROP, relies on carefully orchestrated steps to allocate memory, manipulate process tokens, and then cause it to be executed by creating a new thread in the external process. Below, we outline the specific APIs used and their roles in this process.

CreateToolhelp32Snapshot. This function takes a snapshot of the specified processes, heaps, modules, and threads used by the specified process. This API is used to capture a snapshot of all processes in the system, which is then used to help discover the target process.

VirtualAlloc. This function reserves, commits, or changes the state of a region of memory within the virtual address space of the calling process. It allocates memory in the process where the injection will occur.

Process32First. This function retrieves information about the first process encountered in a system snapshot. It can be used to begin the enumeration of processes captured by CreateToolhelp32Snapshot.

Process32Next. This function retrieves information about the next process recorded in a system snapshot. It continues the enumeration of processes to locate the desired target process.

OpenProcess. This function opens an existing local process object. It gets a handle to the target process to perform further operations such as memory allocation and writing.

OpenProcessToken. This function opens the access token associated with a process. It opens the token of the target process to adjust its privileges, facilitating further actions like memory allocation, changing memory permissions, and code injection.

AdjustTokenPrivileges. This function enables or disables privileges in the specified access token. It adjusts the token privileges of the target process to allow necessary actions like memory allocation and execution.

CreateFileA. This function creates or opens a file or I/O device. This may be used to create or open a file that can later be mapped into the address space of the target process.

NtCreateSection. This native function creates a section object. This section object that can be mapped into the address space of a process.

NtMapViewOfSection. This native function maps a view of a section into the virtual address space of a process. It can map the previously created section into the target process's address space.

VirtualProtectEx. This function changes the protection on a region of committed pages in the virtual address space of a specified process. Unlike with VirtualProtect, you can do this to other processes. It is able to change the memory protection of the newly mapped region to be made executable, allowing the soon to be injected code to be executed.

WriteProcessMemory. This function writes data to an area of memory in a specified process. It is able to inject the shellcode into a memory region of a target process.

CreateRemoteThread. This function creates a thread that runs in the virtual address space of another process. With this created thread in the target process, it can execute the injected shellcode.

6.2 General Flow of the Process Injection

Additional details will follow. A specific implementation of this technique involves creating a new `Urlmon.dll` that is injected into the target process. This new DLL contains the malicious shellcode, which is written to the allocated memory and subsequently executed. This process effectively allows the attacker to inject code inside the victim process and then launch it, using the APIs and native APIs outlined above. The following gives a brief overview of some of the steps taken:

Memory Allocation. `VirtualAlloc` is called to allocate memory to be used for a `PROCESSENTRY32` structure as well as for space in which to build a custom function.

Capturing a Snapshot of Processes. The technique begins by using `CreateToolhelp32Snapshot` to capture a snapshot of all processes.

Custom Function and Process Enumeration. Using the allocated memory, a custom function is created that allows for efficient string comparison, which internally calls `Process32Next`. Together, `Process32First` and `Process32Next` are used within the function to capture the PID of the target application.

Privilege Adjustment. Once the target process is identified, `OpenProcess` is used to get a handle to it. `OpenProcessToken` and `AdjustTokenPrivileges` are then used to modify the process's token, granting the necessary privileges so that potential problems can be avoided.

Section Creation and Memory Mapping. `CreateFileA` may be used to open a file, `Urlmon.dll`, to map into the target process. `NtCreateSection` creates a section object, while `NtMapViewOfSection` maps a view of this section into the external process' address space, where shellcode can be injected.

Shellcode Injection and Execution. `VirtualProtectEx` changes the protection of the newly mapped `Urlmon.dll` within `VUPlayer`. `WriteProcessMemory` is then used to inject the shellcode into this region. Finally, `CreateRemoteThread` starts a thread in the target process, which caused our final payload, the injected shellcode, to execute.

6.3 PROCESSENTRY32: Structure and Its Usage

The `PROCESSENTRY32` structure is needed to list all processes and their Process Identifiers (PIDs). In process injection attacks, acquiring the PID of the target process is utterly essential for any future operations. This structure, thus, is important for the enumeration of processes, allowing all PID's to be identified. Both the PID and the string name for the process occur at fixed offsets, so one can iterate through these, doing string comparison on the name of the process and the target, until it is found. Then, a slight adjustment can be made to capture the PID.

This structure is an example of an important structure that must be produced via ROP – it is initially mostly NULLs, and only a pointer to it need be supplied. Subsequent APIs will initialize it and then populate it with data. The lower-level equivalent of `PROCESSENTRY32`, which can be used for process injection by Windows syscalls, is `SYSTEM_PROCESS_INFORMATION`. Brizendine's presentation at Black Hat Middle East [9] and Hack in the Box Amsterdam 2023 highlights an approach using all Windows Syscalls, the `SYSTEM_PROCESS_INFORMATION`, and doing it all via shellcode.

Process32First: Initializing Process Enumeration. The Process32First API initiates the PROCESSENTRY32 structure. This function needs to be called only once to kick off the process enumeration. Building this structure is relatively straightforward, requiring memory to be setup via ROP. The structure needs to be correctly initialized for the Process32First API to function properly. The requirements are minimal, as it only needs to have the dwSize member of the structure set, and all other members can be set to NULL. The output from a previous VirtualAlloc call will be used to supply the memory for the PROCESSENTRY32 structure. This is done just to avoid any unwelcome problems with the stack, where memory could get overwritten, though with careful planning, the stack would also be suitable.

6.4 Research Problem: No String Comparison via ROP?.

One seemingly insurmountable challenge in process injection is identifying the target binary since string comparison can be extremely difficult in Return-Oriented Programming (ROP). This difficulty arises due to the potential lack of required ROP gadgets. However, we present a unique, universal solution that provides a reliable means of string comparison via ROP. This solution works consistently, allowing a specific process to be pinpointed and injected into using ROP.

Creating an EnumerateProcesses Function in Process Memory. The goal is to find the specific Process ID (PID) from the list of active processes, in this case VUPlayer. To achieve this, we create a custom EnumerateProcesses function that will be placed in process memory. This function iterates through the processes to locate the target string, and then once it does that, it exits out. From there, the PID can easily be found from the current PROCESSENTRY32 structure, as it is at a fixed offset.

Here is Assembly the code used for the EnumerateProcesses function:

```
enumerateProcesses:
mov ebp, edx
push dword ptr [ebp-800h] # lppe - parameter #2 for Process32Next
mov edx, dword ptr [ebp-7FCh] # address of dereferenced WinAPI, Process32Next
push dword ptr [edx] # hSnapshot - parameter #1 for Process32Next
mov edx, dword ptr [ebp-7F8h]
call dword ptr [edx] # Call Process32Next - this is the only place it is called!
mov edi, dword ptr [ebp-800h] # LPPE, pointing to Process32Entry struct
add edi, 24h # Offset to target string of the process's name in Process32Entry struct
mov esi, dword ptr [ebp-7F4h] # string of target application: "VUPlayer"
xor ecx, ecx
mov cl, 8
cld
repe cmpsb
jecxz match
jmp enumerateProcesses
match:
ret
```

Clearly, the function cannot just magically be placed into memory. To create the EnumerateProcesses function, we write its bytes directly into memory using mov dereferences. Before this, the above Assembly can be compiled, and we can obtain its bytes to use to write to memory under our control. Once done, the function will now exist in memory, like any other function. This could be done with RWX memory, or RW memory that is then made RX. There are many possible ways to approach it. The below demonstrates how the custom function is written to memory:

```
# Write the function to memory
0x004016db, # pop ebp # ret # load
```

```

0x00078000, # writable memory under our control
0x004016d9, # pop esi # ret # load
# 0x89D5FFB5, # bytes that correspond to our custom Process Enumerate
Function.
0xB5FFD589,
0x004016c3, # mov dword ptr [ebp], esi # ret
0x0040132f, # inc ebp # ret# Repeat inc ebp and other instructions to com-
plete the function

```

Once the custom EnumerateProcesses function identifies the target process by matching the name, it accesses another field in the PROCESSENTRY32 structure to then retrieve the PID.

By creating and deploying a custom EnumerateProcesses function in memory, we solve the challenge of string comparison in ROP-based process injection. String comparison certainly could be done without it, but it could be challenging in some cases to find and set up the proper gadgets. This seems to provide a highly efficient solution to the problem, one which will always work, so long as other basic APIs or the deeper, lower-level, native equivalents can work. Thus, this method allows for the reliable identification and targeting of specific processes. Without this, process injection would be infeasible from a purely ROP standpoint, as you could not do string comparison. We experimented with randomly injecting into unknown processes, to simply avoid this problem, but the problem was that when you come to the first processes in the PROCESSENTRY32, the first several are related to System or are just not injectable. This would require calling Process32Next numerous times to find one random one that would be injectable. Though doing this would make this approach much less relevant, as you could not specify a target process to inject into.

6.5 OpenProcess and Beyond

The OpenProcess API allows us to convert a PID to a usable handle—we cannot proceed further without it. In terms of utilizing patterns, it requires a combination of pushad and mov dereferences. The dwProcessId parameter, which needs to be set to the PID obtained from the Process32Next call, is needed for getting a handle to the target process, in this case, VUPlayer. The specific pattern for OpenProcess has several variations for the pushad segment. After executing pushad, mov dereferences handle the remaining parameters.

OpenProcessToken: Accessing the Process Token. The OpenProcessToken WinAPI is used to get an access token associated with a process, which is needed for doing privilege adjustments. This WinAPI is implemented using just mov dereferences. The TokenHandle parameter, which is needed for future privilege adjustments, is made by storing a pointer to the token in memory. This handle can later be used to modify the process's privileges.

AdjustTokenPrivileges: Modifying Process Privileges. To avoid some complications that can arise when trying to do process injection, it is often necessary to adjust the process's privileges. The AdjustTokenPrivileges WinAPI achieves this by modifying the access token given by OpenProcessToken. In terms of ROP, this API is implemented entirely using mov dereferences. There are complex nested structures required for this WinAPI. The TOKEN_PRIVILEGES structure, which includes the LUID_AND_ATTRIBUTES for some privileges, needs to be accurately built in memory via ROP. As such, it is wise to carefully plan it out in advance. By setting the correct privilege attributes, like SE_DEBUG_PRIVILEGE, the process gets powerful permissions, so that future problems and difficulties can be avoided. Without doing this, sometimes – depending on the privileges of the originating process – there can be lots of failures, when trying to do certain tasks associated with process injection. That is to say WinAPIs will return failures in EAX and native APIs will record the an NTSTATUS code that corresponds with failure. Having these additional privileges helps provide the best opportunity for success.

Creating and Mapping a Section for Process Memory. For our injection, a common DLL, such as Urlmon.dll, must be mapped into the target process's memory. This involves a series of APIs including CreateFileA, NtCreateSection, NtMapViewOfSection, VirtualProtectEx, WriteProcessMemory, and CreateRemoteThread. For the sake of brevity, these will all be addressed here. CreateFileA opens the file Urlmon.dll with the necessary access rights using the mov dereference technique. NtCreateSection is a native API that creates a section object for the DLL, setting appropriate attributes like SEC_IMAGE and PAGE_READONLY. These more limited settings can

help avoid problems, though other attributes can be used depending on the binary and its privileges. `NtMapViewOfSection` is another native API, and this maps a view of the section into the address space of `VUPlayer`, the target process, providing a base address that we will use later. `VirtualProtectEx` allows us to take the handle to `VUPlayer` we obtained from `OpenProcess` and the base address from `NtMapViewOfSection`, allowing us to change this section of code in `VUPlayer` to `PAGE_EXECUTE_READWRITE`. This is needed for code execution. `WriteProcessMemory` lets us write or inject the shellcode into `VUPlayer`'s memory, from our originating process. At this point all the shellcode has been placed in the external process, and we can then cause it to execute with `CreateRemoteThread`. This creates a thread in the target process to execute the injected shellcode.

Another simpler approach could be to do `NtCreateSection` in the originating process, make changes there, and then simply do `NtMapViewOfSection` and `CreateRemoteThread` in the external process. Other variations are possible.

7 Concluding Remarks

As part of this research, four process injection techniques were done via ROP, two of them being only minor variations. The goal was to demonstrate that highly complex actions such as this could be done in Windows via ROP. One significant problem, that of not being able to do strings comparison for process enumeration, was overcome with a unique solution, one which could have far-reaching implications for solving other minor or not minor problems within the context of ROP, where certain actions – in terms of more complex actions done via Assembly – may be not be common enough to be predictable or found in many binaries, thus making them largely infeasible due to what likely would be a limited attack surface. Certainly, on individual binaries, appropriate sets of gadgets could be found to facilitate strings comparison, but it would be far from universal. This research greatly simplifies this.

Our objective was not to merely enumerate the various process injection techniques, as those topics have been extensively covered in previous discussions. Instead, we aimed to provide a methodology for achieving process injection purely via ROP, without relying on higher-level languages. By focusing on the practical aspects of ROP-based process injection, we have highlighted the potential and flexibility of doing a pure ROP approach to executing sophisticated attacks, which we classify as shellcodeless ROP, even though – ironically enough – we have a miniature shellcode that we inject into another process and execute, which nominally pops a calculator. In the longest of the process injections we did, in all 29 WinAPIs and native APIs were executed, dealing with highly complex parameters, including nested structures, which had to be built by ROP. This, likely, is probably the most complex and longest ROP chain built for Windows – at least from what we are familiar with. In a sense, this serves as a proof of concept for the sheer capabilities of ROP.

One of the most significant contributions of this paper is the novel concept of building an `EnumProcesses` function. When ROP falls short to the needs of process injection – i.e. requiring more complex string comparison – we do not just give up, but we find a unique, universal solution – writing a custom function to memory and making it executable as its own API, essentially. This approach could lend itself to other process injection techniques or other more advanced Windows ROP attacks, where you need to overcome some limitations.

The ROP patterns we developed, as provided in the slides, do a lot of the heavy lifting needed to allow for ROP usage of some of these APIs, avoiding potentially significant labor in needing to develop them from scratch. As always, when possible, a `pushad`-based approach to ROP will be more efficient, and this helps streamline the usage of some of these.

References

- [1] O. Chechik, "Banking Trojan Techniques: How Financially Motivated Malware Became Infrastructure," 2022. [Online]. Available: <https://unit42.paloaltonetworks.com/banking-trojan-techniques/>.
- [2] A. Mazor, "StackBombing - Next Gen Process Injection," Digital Whisper, [Online]. Available: <https://www.digitalwhisper.co.il/files/Zines/0x77/DW119-1-StackBombing.pdf>.
- [3] A. Klein and I. Kotler, "Windows Process Injection in 2019," in *Black Hat*, 2019.

- [4] O. Yair, "ROP - From Zero to Nation State in 25 Minutes," in *BsidesTLV*, 2020.
- [5] D. C. D'Elia, L. Invidia and L. Querzoni, "Rope: Covert Multi-Process Malware Execution with Return-Oriented Programming," in *Computer Security-ESORICS 2021: 26th European Symposium on Research in Computer Security*, 2021.
- [6] B. Brizendine and S. Kusuma, "Advanced ROP Framework: Pushing ROP to Its Limits," in *DEF CON 31*, Las Vegas, NV, 2023.
- [7] B. Brizenidne and A. Babcock, "Shellcodeless JOP:Advanced Code-Reuse Attacks," in *Black Hat Middle East and Africa*, 2021.
- [8] B. Brizendine, "ROP ROCKET," GitHub, [Online]. Available: https://github.com/Bw3ll/ROP_ROCKET.
- [9] B. Brizendine, "ShellWasp and Offensive Usage of Windows," in *Black Hat Middle East and Africa*, 2022.